Fisher-Kolmogorov
Equation for neurodegenerative diseases

Lorenzo Esposito 10765981, Francesco Grassi 10841139,
Giorgio Venezia 10807335

August 24, 2025

Version 1.1.0

Contents

1 Introduction

Neurodegenerative diseases such as Alzheimer's, Parkinson's, and ALS involve progressive neuronal death caused by misfolded proteins that spread through brain tissue. These diseases follow a prion-like mechanism in which pathogenic proteins convert normal proteins into toxic aggregates, creating cascading propagation across neural networks.

Mathematical modeling uses reaction-diffusion equations to capture this spreading process. The models treat protein aggregation as a non-linear reaction coupled with anisotropic diffusion through brain tissue, reducing the biology mechanism to equations governing protein concentration dynamics in space and time.

This project implements a physics-based reaction-diffusion model combining the Fisher-Kolmogorov equation with directional diffusion terms inspired by the framework presented in the article [1]. The model simulates growth and diffusion in terms of concentration of misfolded protein.

If these models are accurate, they can predict disease progression patterns, identify vulnerable brain regions, and inform therapeutic strategies targeting early-stage protein aggregation before irreversible neuronal damage occurs.

2 Problem model definition

The problem in his mathematical form

$$\begin{cases} \dfrac{\partial c}{\partial t} - \nabla \cdot (D \cdot \nabla c) - \alpha c(1-c) = 0 & \text{in } \Omega, \\ D \cdot \nabla c \cdot \vec{n} = 0 & \text{in } \partial\Omega, \\ c(t=0) = c_0 & \text{in } \Omega. \end{cases} \tag{2.1}$$

2.1 Kinetics of prion-like spreading

To describe the fundamental mechanism behind prion-like spreading, the article [1] gives us a simplified kinetic model, which distinguishes between two states of a protein: the natural healthy form p and the misfolded, pathogenic form $\tilde{p}$. The misfolded proteins propagate through the brain by converting healthy proteins into their misfolded form, like autocatalytic behavior.
In the model are considered some kinetic processes:

- **Recruitment and conversion:** Healthy proteins bind to misfolded proteins and adopt the misfolded state at a rate k_{12}: $p + \tilde{p} \overset{k_{12}}{\rightarrow} \tilde{p} + \tilde{p}$.
- **Fragmentation:** Misfolded proteins can fragment to generate new infectious seeds (although we do not consider this in the final formulation).
- **Production and clearance:** Healthy proteins are continuously produced at a constant rate k_0 and cleared at rate k_1, while misfolded proteins are cleared at a separate rate $\tilde{k_1}$.
- **Diffusion:** Both types of proteins diffuse in the surrounding medium, characterized by diffusion coefficients D_p and $D_{\tilde{p}}$ respectively.

The model leads to a system of coupled PDEs governing the evolution of the concentrations of the two different types of proteins:

$$\frac{dp}{dt} = Div(D_p \nabla p) + k_0 - k_1 p - k_{12} p\tilde{p} \tag{2.2}$$

$$\frac{d\tilde{p}}{dt} = Div(D_{\tilde{p}} \nabla \tilde{p}) - \tilde{k_1}\tilde{p} + k_{12} p\tilde{p} \tag{2.3}$$

To simplify the system, the article assumes that the concentration of healthy protein is significantly higher than that of the misfolded form (i.e., $p \gg \tilde{p}$), allowing

a quasi-steady-state approximation for p. Under this assumption the healthy protein concentration is defined as:

$$p \approx \frac{k_0}{k_1 + k_{12}\tilde{p}} \tag{2.4}$$

And expanding this expression in a Taylor series around $\tilde{p} = 0$, it is substituted into the equation for $\tilde{p}$, yielding a nonlinear reaction-diffusion equation.

To further simplify the model, it is introduced a normalized concentration for misfolded proteins, which is:

$$c = \frac{\tilde{p}}{\tilde{p}_{max}},\ 0 \leq c \leq 1. \tag{2.5}$$

This leads to a logistic-type equation:

$$\frac{\partial c}{\partial t} = \mathrm{Div}(D\nabla c) + \alpha c(1 - c) \tag{2.6}$$

where $D = D_{\tilde{p}}$ is the diffusion tensor and α is the effective growth rate defined as:

$$\alpha = \frac{k_{12}k_0}{k_1} - \tilde{k}_1 \tag{2.7}$$

The Equation 2.7 captures the key dinamics of prion-like propagation: the diffusion of misfolded proteins and their growth due to conversion of healthy proteins. The growth rate α increases with the conversion rate k_{12} and the equilibrium healthy protein concentration $p_0 = k_0/k_1$, and decreases with the clearance rate of misfolded proteins $\tilde{k}_1$.

2.2 Fisher–Kolmogorov

To model the physics of prion-like diseases, we adopt the Equation 2.1, which is known in mathematical biology as the Fisher–Kolmogorov equation. To have a numerical description of the problem, we need to solve the Equation 2.1 with respect to the concentration c.

2.3 Weak formulation

As for the most PDEs, the first step to take is to build the weak formulation.

1. To derive the weak formulation, it's introduced the following Hilbert space

$$H^1(\Omega) = \left\{ v \in L^2(\Omega) \,:\, \nabla v \in \left[L^2(\Omega)\right]^d \right\},$$

   It's required that $c \in H^1(\Omega)$ in order to be solution of Equation 2.1.

2. For function $c \in H^1(\Omega)$ to be solution of Equation 2.1 it's required that $\forall v \in H^1(\Omega)$:

$$\int_\Omega \frac{\partial c}{\partial t} \cdot v \; dx - \int_\Omega \nabla \cdot (D \cdot \nabla c) \cdot v \; dx - \int_\Omega \alpha c(1 - c) \cdot v \; dx = 0 \tag{2.8}$$

3. By integrating by parts the second term,$-\int_\Omega \nabla \cdot (D \cdot \nabla c) \cdot v \; dx$, the following expression is obtained:

$$-\int_\Omega \nabla \cdot (D \cdot \nabla c) \cdot v = \int_\Omega D \cdot \nabla c \nabla v - \int_{\partial\Omega} D \cdot c \cdot v \cdot \vec{n} \tag{2.9}$$

Here the term $\int_{\partial\Omega} D \cdot c \cdot v \cdot \vec{n} = 0$ due to the second equation of Equation 2.1. By substituting in the Equation 2.8, we obtain the weak formulation of Equation 2.1

$$\int_\Omega \frac{\partial c}{\partial t} \cdot v \; dx + \int_\Omega D \cdot \nabla c \cdot \nabla v \; dx - \int_\Omega \alpha c(1-c) \cdot v \; dx = 0 \tag{2.10}$$

2.4 Time discretization

As in Equation 2.10 there's the term $\int_\Omega \partial_t c \cdot v \; dx$, it's necessary to discretize the time derivative. We've decided to use the Backward Euler Method instead of Forward Euler Method or Mid-Point Method because it's unconditionally stable, while FWE is not, and because it's less computationally expensive than MP. To use this method we considered the interval $[0, T]$, where T is the final simulation time, and we divided it into N equal subintervals of length $\Delta t = \frac{T}{N}$. For each interval t^{n+1} we can approximate the time derivative as:

$$\left.\frac{\partial c}{\partial t}\right|_{t^{n+1}} \approx \frac{c^{n+1} - c^n}{\Delta t}$$

which leads to the time-space discretized expression:

$$\int_\Omega \frac{c^{n+1} - c^n}{\Delta t} \cdot v \; dx + \int_\Omega D \cdot \nabla c^{n+1} \cdot \nabla v \; dx - \int_\Omega \alpha c^{n+1}(1 - c^{n+1}) \cdot v \; dx = 0 \tag{2.11}$$

2.5 Linearization

It's still not possible to utilize Equation 2.11 for the numerical solution because it presents a non-linear term in $-\int_\Omega \alpha c(1-c) \cdot v \; dx$. To solve the problem, we need to used a linerized form of the Equation 2.11. It's possible to do so using the Fréchet derivative. For a given functional $G : V \to R$, the Fréchet derivative at a point $u \in H^1(\Omega)$ is a linear operator $A(u) = \frac{dG}{du} : V \to R$ such that:

$$lim_{\|d\| \to 0} \frac{|R(u+d) - R(u) - A(u)d|}{\|d\|} = 0$$

Now we can define the nonlinear residual:

$$\begin{aligned} R(c^{n+1} + \delta, v) = & \int_\Omega \frac{c^{n+1} + \delta - c^n}{\Delta t} \, v \, dx + \int_\Omega D\nabla(c^{n+1} + \delta) \cdot \nabla v \, dx \\ & - \int_\Omega \alpha(c^{n+1} + \delta)(1 - c^{n+1} - \delta) \, v \, dx \end{aligned}$$

Now, if we take $R(c^{n+1}+\delta, v) - R(c+\delta, v)$ we obtain:

$$\begin{aligned} R(c^{n+1}+\delta, v) - R(c^{n+1}, v) = & \left(\int_\Omega \frac{c^{n+1}+\delta-c^n}{\Delta t}\, v\, dx - \int_\Omega \frac{c^{n+1}-c^n}{\Delta t}\, v\, dx\right) \\ & + \left(\int_\Omega \delta\nabla(c^{n+1}+\delta)\cdot\nabla v\, dx - \int_\Omega \delta\nabla c^{n+1}\cdot\nabla v\, dx\right) \\ & - \left(\int_\Omega \alpha(c^{n+1}+\delta)(1-c^{n+1}-\delta)v\, dx - \int_\Omega \alpha c^{n+1}(1-c^{n+1})v\, dx\right) = \\ & \int_\Omega \frac{\delta}{\Delta t}\, v\, dx + \int_\Omega D\nabla\delta\cdot\nabla v\, dx - \int_\Omega \alpha\left(\delta(1-2c^{n+1}) - \delta^2\right) v\, dx \end{aligned}$$

We can approximate $(\delta)^2$ as a higher-order small term, leading to the following expression:

$$\begin{aligned} R(c^{n+1}+\delta, v) - R(c^{n+1}, v) = & \int_\Omega \frac{\delta}{\Delta t}\, v\, dx + \int_\Omega D\nabla\delta\cdot\nabla v\, dx \\ & - \int_\Omega \alpha\delta(1-2c^{n+1})\, v\, dx + \mathcal{O}\left((\delta)^2\right) \end{aligned}$$

The right-hand side of section 2.5 with exception of $\mathcal{O}\left((\delta)^2\right)$ corresponds to the action of the Fréchet derivative:

$$\frac{dR}{dc}(c^{n+1})(\delta, v) = a(c^{n+1})(\delta, v)$$

Now, solving the Equation 2.11 it's equivalent to find $c^{n+1} \in H^1(\Omega)$ such that $R(c^{n+1}, v) = 0,\ \forall v \in H^1(\Omega)$. To solve this we use the Newton's method: given an initial guess $c^{(0)}$,

while $k = 1, 2, ..., N$ *and not convergent* **do**
  find $\delta^{(k)}$ *s.t* $a(c^{(k)})(\delta^{(k)}, v) = -(c^{(k)}, v)\ \forall v \in H^1(\Omega)$
  $u^{(k+1)} = u^{(k)} + \delta^{(k)}$
end while

The problem $a(c^{(k)})(\delta^{(k)}, v) = -R(c^{(k)}, v)$ is a linear differential problem and can be solved used finite elements.

2.6 Finite element setting and final assembly

After a linear differential form of the original problem in Equation 2.1 is find in the form of $a(c^{(k)})(\delta^{(k)}, v) = -R(c^{(k)}, v)$, we can discretize this problem in space using the Galerkin finite element method. To do so, we consider a vector space $V_h \subset H^1(\Omega)$ which dimension are bounded and equals to N_h, our aim is to find $c_h^{(k)}$ solution of the problem $a(c_h^{(k)})(\delta^{(k)}, v_h) = -R(c_h^{(k)}, v_h)\ \forall v_h \in H^1(\Omega)$. We introduce the bases of V_h: $\{\phi_j\}_{j=1}^N$, now it's possible to write:

$$c_h^{(k)}(x) = \sum_{j=1}^N c_j^{(k)}\phi_j(x), \qquad \delta^{(k)}(x) = \sum_{j=1}^N \delta_j^{(k)}\phi_j(x).$$

and substitute these expressions in the problem considering the test functions $v = \phi_i$ and obtaining

$$a\left(\sum_{j=1}^{N} c_j^{(k)}\phi_j(x)\right)\left(\sum_{j=1}^{N}\delta_j^{(k)},\phi_i\right) = -R\left(\sum_{j=1}^{N} c_j^{(k)}\phi_j(x),\phi_i\right) \tag{2.12}$$

After some computation, the following result is obtained:

$$\left(\frac{M}{\Delta t} + K - J\left(c_k^{n+1}\right)\right)\cdot\delta^{(k)} = -M\cdot\frac{c_k^{n+1}-c^n}{\Delta t} - K\cdot c_k^{n+1} + N\left(c_k^{n+1}\right) \tag{2.13}$$

where:

- $\delta^{(k)}$ is the vector of unknown for this problem;
- c_k^{n+1} is the value of the concentration calculated at the previous newton interaction k;
- c^n is the value computed at the previous timestamp n;
- $M = (m_{ij}) = \int_\Omega \phi_i\phi_j\ dx$;
- $K = (k_{ij}) = \int_\Omega D\cdot\nabla\phi_i\cdot\nabla\phi_j\ dx$;
- $J\left(c_k^{n+1}\right) = (j_{ij}) = \int_\Omega \alpha(1-2c_k^{n+1})\phi_i\phi_j\ dx$
- $N\left(c_k^{n+1}\right) = (n_{ij}) = \int_\Omega \alpha\cdot c_k^{n+1}\cdot(1-c_k^{n+1})\phi_i\ dx$

To be more clear we introduce the matrix LHS and the vector RHS defined as:

$$LHS = \frac{M}{\Delta t} + K - J\left(c_k^{n+1}\right), \qquad RHS = -M\cdot\frac{c_k^{n+1}-c^n}{\Delta t} - K\cdot c_k^{n+1} + N\left(c_k^{n+1}\right).$$

At each timestamp $n+1$, Newton method is used to find $\delta^{(k)}$ by solving the linear system in Equation 2.13, which is then used to compute the solution for the timestep c_{n+1}. At each iteration of the Newton Method both LHS matrix and RHS vector are updated using the new solution c_{n+1}.

3 Set up and computation

The problem has been solved using `C++` combining `deal.ii` library and `MPI` for parallelization purpose.

3.1 1D and 3D: Key aspects

The problem was initially solved in 1D as a proof of concept and to check feasibility, then scaled to 3D.

3.1.1 The mesh

The mesh used for 3D computation is read from an external *.msh* file provided by the original author of the article [1]. The implementation distinguishes between different brain tissue types through material IDs, with the code specifically tracking Grey Matter (material ID 1), White Matter(material ID 2). A material mapping system associates each cell with its corresponding tissue type, enabling tissue-specific parameter assignment.

In 1D, the mesh is simply an interval $[0, l] \in \mathbb{R}$ divided into N equal subintervals. For our computation, we have chosen $l = 2$ and $N = 200$.

3.1.2 Diffusion Term

For the implementation in 3D, the terms in Equation 2.1 indicated as D is a tensor, called Diffusion matrix defined as

$$\mathbf{D} = d^{\text{ext}}\mathbf{I} + d^{\text{axn}}\mathbf{n} \otimes \mathbf{n}$$

where the isotropic extracellular d^{ext} component is combined with an anisotropic axonal d^{axn} transport component. The fiber direction vector $\mathbf{n}$ is obtained through a `Diffusion` class that supports three different fiber orientation models via the `type_of_diffusion` parameter: axonal, circumferential, radial.

In 1D implementation, instead, this term is simply a scalar $D = d \in \mathbb{R}$.

3.1.3 Similarity

The methods used to solve equation had to be keeped unchanged, indeed the core structure of the algorithm stays unaltered

- Newton's method to solve nonlinear systems.

- Time-stepping stucture with backward Euler discretization.
- GMRES with preconditioner as linear solver.

3.1.4 Differences

- 1D implementation uses a less complex and uniform mesh built with

  `subdivided_hyper_cube()`.

  while 3D implementation reads the mesh directly from the proper file.

- In 1D the diffusion coefficient is just a scalar:

  ```
  const double diffusion_coefficient_loc = diffusion_coefficient.value();
  ```

  while in 3D we model an anisotropic diffusion with tensor formulation:

  ```
  const Tensor<2, dim> D = d_ext_matrix + d_axn_loc * normal_matrix;
  ```

- In 1D we assume uniform material properties throughout the domain, while the 3D program handles multiple brain tissue types (grey matter, white matter) through a gradient of α.

3.1.5 Parameters

The choice of the parameter is crucial for convergence, especially for the 3D simulation. In this case, the inputs and the parameters of the problem are:

1. **T**: final time of the simulation.
2. Δt: time step for time discretization.
3. **Mesh**: A 3D discretized representation of the domain in $\mathbb{R}^3$.
4. **Degree**; the polynomial degree of the finite elements used to construct the Finite Element Method.
5. **d_ext**: Isotropic extracellular diffusion coefficient.
6. **d_axn**: Anisotropic axonal transport along axonal fiber directions.
7. **Diffusion type**: Defines the direction of diffusion of the misfolded proteins and could be:
   - Axonal
   - Circumferential
   - Radial

   These types of diffusion are also taken in account by the article [1].

8. α; growth rate of misfolded protein concentration.

9. **Initial seed**: initial value of the non-zero concentration.

10. **Seed Region**; subdomain in which the initial concentration has the non-zero value Initial Seed. In our 3D simulations, we actually pass a point x_0 and the program computes a spherical region that has the point x_0 as center.

3.2 Implementation

3.2.1 Implementation in 1D

To verify the correctness of our approach, we decided to implement a 1D solver as done by the author of the article [1] and then to compare the results. In this case, the domain considered is the interval $[-1, 1] \subset \mathbb{R}$, the diffusion term D reduces to a scalar parameter d and the initial seed region consists of a single point at the center of the domain ($x = 0$).

The mesh is created in the interval with $N = 200$ mesh elements and degree = 1. After initializing all necessary objects as done in the laboratory sessions, the method `solve()` executes the system resolution for each time step and writes the output for that instant into a `.vtu` file.

The method `solve()` itself calls the function `solve_newton()`, which performs Newton iterations to solve the non-linear system. For each iteration, the system is linearized around the current solution via `assemble_system()`, and the resulting linear system is solved using `solve_linear_system()`. This process repeats until the norm of the residual vector falls below a fixed threshold ($10^{-6} \cdot \|\mathbf{r}\|$ in our code, where r is the residual vector).

Assemble_system() follows the same structure used in the laboratory exercises to compute the LHS matrix and the RHS vector. These terms represent the linearized versions of the original system, which we obtain using the values of the current and previous solutions to compute the relevant quantities and the parameters.

Since homogeneous Neumann boundary conditions are forced to be null and the Dirichlet condition are not present, there is no need to handle boundary conditions explicitly.

Solve_linear_system() assumes that the LHS matrix and RHS vector are correctly assembled. We use the Generalized Minimal Residual Method (GMRES) method to numerically solve the resulting linear system and the SSOR preconditioner is applied to accelerate convergence.

3.2.2 Implementation in 3D

The primary differences in the 3D solver involve the use of parallelization, the use of different parameters as explained in subsection 3.1.5 and the presence of two different materials.

Parallelization

The 3D solver, due to the large number of nodes in 3D meshes and the nonlinear nature of the problem, results in exponentially higher complexity. This makes parallelization essential for keeping the computation feasible. Our parallelization approach leverages Trilinos with MPI to distribute the computational workload. We partition the mesh across multiple processes, with each process handling its assigned degrees of freedom while maintaining communication at partition boundaries. This domain decomposition strategy, combined with efficient load balancing, helps abstract much of the parallel complexity and enables scalable performance.

Parameters representation

- **Mesh**: as already mentioned in subsection 3.1.5, the mesh is represented in into an external `.msh` file which for each finite element shows also the corresponding material ID which can be either 1 for Grey Matter or 2 for White Matter.
- **Piece-wise function**: as mentioned in paper [1] the terms d^{axn}, d^{ext} and α are assumed as piecewise constant function depending on the material ID. This behaviour is implemented using a `std::vector<double>` where each component represents the value that the parameters have in each material ID, where the first component refers to material ID 1 and the second to material ID 2. In order to make use of it in the solver, we request a parameter `const unsigned int component` in the `virtual double value()` function alongside the point to valuate. The value of the `component` parameter has to be either 1 or 2 to distinguish a point in Grey Matter or in White Matter.
- **Initial Seed**: computes the initial seed region as the intersection of a spherical area centered at the given seed point and the domain mesh.

In the 3D implementation we switched from a SSOR preconditioner to an ILU preconditioner.

4 Performance evaluation

4.1 Different parameters

4.2 Convergence

5 Final results

Bibliography

[1] J. Weickenmeier, M. Jucker, A. Goriely, and E. Kuhl, "A physics-based model explains the prion-like features of neurodegeneration in alzheimer's disease, parkinson's disease, and amyotrophic lateral sclerosis," *Journal of the Mechanics and Physics of Solids*, vol. 124, pp. 264–281, 2019. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0022509618307063